Security Scan Summary

Target: **orangehrm/orangehrm** · branch: **main**
Scan run: 20261106 (3)
Status: **PERLU DIPERBAIKI**

5	38	75	180
CRITICAL	HIGH	MEDIUM	LOW

KETERANGAN

JENIS SCAN	SCOPE & REMEDIATION	COMMAND
SCA Snyk Open Source	Software Composition Analysis: the dependency tree from the manifest/lockfile (package.json, pom.xml, etc.) is matched against a vulnerability database (CVE). **Remediation:** upgrade to the version in "Fixed in" after checking the upgrade path on snyk.io. For a **transitive dependency** (see "Introduced through"), upgrade the direct dependency that pulls it in. Pick the nearest safe version, check for breaking changes, then regression test. If there is no fix / it is not exploitable, mitigate or ignore with a documented justification.	`snyk test --all-projects`
SAST Snyk Code	Static Application Security Testing: the source code is analyzed with data-flow / taint analysis for vulnerability patterns (SQL injection, XSS, path traversal, hardcoded secrets). **Remediation:** validate the source → sink data flow at the reported location (it can be a false positive), then fix it, e.g. parameterized queries, input validation, output encoding. If it is a false positive, ignore with a documented justification.	`snyk code test`
GITLEAKS local scan	Secret scanning (regex + entropy) for passwords, tokens, API keys, and private keys across **the entire git history**, including commits that have since been removed from HEAD. Secret values are redacted. **Remediation:** treat as compromised: rotate/revoke first, then move the secret into an environment variable / secret manager.	`gitleaks git --redact`
TRIVY local scan	*Repos with a Dockerfile.* IaC misconfiguration scanning: the Dockerfile is evaluated against security best-practice checks, e.g. container running as root, unpinned base image (:latest), no HEALTHCHECK. **Remediation:** follow the "Resolution" on each finding.	`trivy config .`

SEVERITY	
C	**Critical**
H	**High**
M	**Medium**
L	**Low**

STATUS REPO	
PERLU DIPERBAIKI	There is a Critical/High/Medium finding, or a secret.
AMAN	No Medium+ finding and no secret found.
GAGAL SCAN	A scan could not complete; coverage is incomplete, not proven safe.

ISTILAH PER TOOL

Glossary of terms and fields that appear in each tool's detail PDF.

JENIS SCAN	Term	Meaning
ALL SCANS	**Severity (C / H / M / L)**	Critical / High / Medium / Low, usually derived from the CVSS score or rule severity. Low findings are still listed in the detail PDF.
	CVE / CWE	**CVE** = public identifier for one specific vulnerability in a given software version. **CWE** = weakness category (e.g. CWE-89 SQL Injection, CWE-22 Path Traversal, CWE-798 Hard-coded Credentials).
	False positive	A finding that is not actually exploitable in the application's context. Must be validated manually before being ignored, and the ignore must include a justification.
SCA Snyk Open Source	**Introduced through**	The dependency path from the project to the vulnerable package. More than one level = **transitive dependency**; remediate by upgrading the **direct dependency** that pulls it in.
	Fixed in	Minimum package version that already contains the patch. Empty = no fix available (mitigate or switch library).
	CVSS score	Base score 0.0–10.0 from the CVSS v3.x/v4 standard.
	Exploit maturity	Public exploit availability: No Known Exploit, Proof of Concept, Functional, Mature. The more mature, the higher the likelihood of exploitation.
SAST Snyk Code	**Location / data flow**	The vulnerable file:line and the taint flow **source** → **sink** (untrusted input reaching a dangerous operation without sanitization).
	Rule	Snyk Code rule ID that triggered, e.g. javascript/Sqli, go/PT (Path Traversal).
GITLEAKS local scan	**Rule / Entropy**	**Rule** = the detector that matched (e.g. generic-api-key, private-key, jwt). **Entropy** = Shannon entropy of the matched value; the higher it is, the more likely it is a real secret (not a placeholder).
	First introducing commit	Commit, author, and timestamp when the secret first entered the history, for investigation and to determine the exposure window.
TRIVY local scan	**Pass / Fail**	Number of Dockerfile policy checks satisfied (PASS) and violated (FAIL).
	Check ID (DS-xxxx)	Trivy policy ID for Dockerfiles, e.g. DS-0001 unpinned :latest, DS-0002 run as root, DS-0026 no HEALTHCHECK.
	File-level finding	A finding with no line number because the violation is a missing instruction (e.g. USER, HEALTHCHECK).

orangehrm/orangehrm branch: main	PERLU DIPERBAIKI	5 C	38 H	75 M	180 L

JENIS SCAN	PROJECT	ISSUES
SCA Snyk Open Source	orangehrm	5 C 36 H 64 M 4 L
SAST Snyk Code	orangehrm	0 C 1 H 11 M 175 L
GITLEAKS local scan	orangehrm	358 Secret
TRIVY local scan	orangehrm	0 C 1 H 0 M 1 L